

Salvador cardoso

The Abstract syntax Grammar for the If-else statement is

$$\langle \text{stmt} \rangle \rightarrow i \langle \text{expr} \rangle t \langle \text{stmt} \rangle \mid i \langle \text{expr} \rangle t \langle \text{stmt} \rangle e \langle \text{stmt} \rangle \mid a$$

$$\langle \text{expr} \rangle \rightarrow b$$

Here, i , t , & e stand for 'if, else, then'; $\langle \text{expr} \rangle$ & $\langle \text{stmt} \rangle$ stand for "conditional expression" & "statement"

- Is this grammar suitable for top-down parsing? Illustrate w/ an example where ambiguity can arise
- Left factor this grammar
- Draw a parse table for the modified grammar. Is there an entry containing multiple productions, leading to ambiguity? How can the ambiguity be eliminated during parsing? What is the implication for the additional rule?
- Write the pseudocode for recursive implementation of LL(1) grammar

a) No, the grammar is ambiguous. It is also

$$\begin{array}{c} \langle \text{stmt} \rangle \\ \swarrow \quad \searrow \\ i \langle \text{expr} \rangle t \langle \text{stmt} \rangle \quad \langle \text{stmt} \rangle \\ \swarrow \quad \searrow \\ i \langle \text{expr} \rangle t \langle \text{stmt} \rangle e \langle \text{stmt} \rangle \end{array}$$

$$\begin{array}{c} \langle \text{stmt} \rangle \\ \swarrow \quad \searrow \\ i \langle \text{expr} \rangle t \langle \text{stmt} \rangle e \langle \text{stmt} \rangle \quad \langle \text{stmt} \rangle \\ \swarrow \quad \searrow \\ i \langle \text{expr} \rangle t \langle \text{stmt} \rangle \end{array}$$

$$i \langle \text{expr} \rangle t i \langle \text{expr} \rangle t \langle \text{stmt} \rangle e \langle \text{stmt} \rangle = i \langle \text{expr} \rangle t i \langle \text{expr} \rangle t \langle \text{stmt} \rangle e \langle \text{stmt} \rangle$$

$$\begin{aligned} \langle \text{stmt} \rangle &\rightarrow i \langle \text{expr} \rangle t \langle \text{stmt} \rangle \langle \text{stmt} \rangle' \mid a \\ \langle \text{stmt} \rangle' &\rightarrow \epsilon / e \langle \text{stmt} \rangle \end{aligned}$$

① $\langle \text{stmt} \rangle \rightarrow i \langle \text{expr} \rangle t \langle \text{stmt} \rangle \langle \text{stmt} \rangle'$	first	i	e	t	a	b	$\$$
② $\langle \text{stmt} \rangle \rightarrow a$	$\langle \text{stmt} \rangle$	①			②		
③ $\langle \text{stmt} \rangle' \rightarrow \epsilon$	$\langle \text{stmt} \rangle'$						
④ $\langle \text{stmt} \rangle' \rightarrow e \langle \text{stmt} \rangle$	$\langle \text{stmt} \rangle'$		④/③				③
⑤ $\langle \text{expr} \rangle \rightarrow b$	$\langle \text{expr} \rangle$					⑤	

$$\text{follow}(\text{stmt}) = \{\$, e\}$$

$$\text{follow}(\text{stmt}') = \{\$, e\}$$

There is an additional entry in $m[\text{'stmt'}, \epsilon]$. As mentioned in class, this ambiguity can be resolved by the parser by greedily consuming input, instead of choosing to take the ϵ -derivation

$\{\text{stmt}\} \rightarrow i \{\text{expr}\} \epsilon \{\text{stmt}\} \epsilon \{\text{stmt}\} \epsilon \mid a$
 $\{\text{stmt}\} \epsilon \rightarrow \epsilon$
 $\{\text{stmt}\} \epsilon \rightarrow \epsilon \{\text{stmt}\} \epsilon$
 $\{\text{expr}\}$

```

d) parseStmt() {
    lookahead = getLookAhead();
    if (lookahead == 'i') {
        match('i');
        parseExpr();
        match('\epsilon');
        parseStmt();
        parseStmtP();
    } else if (lookahead == 'a') {
        match('a');
    } else {
        error();
    }
}

```

```

parseStmtP() {
    lookahead = getLookAhead();
    if (lookahead == '\epsilon') {
        return;
    } else if (lookahead == '\epsilon') {
        match('\epsilon');
        parseStmt();
    } else {
        error();
    }
}

```

```

parseExpr() {
    lookahead = getLookAhead();
    if (lookahead == 'b') {
        match('b');
    } else {
        error();
    }
}

```


2) consider the Abstract syntax grammar

$$S \rightarrow S(S)S \mid \epsilon$$

a) Is this grammar suitable for left-recursive parsing? If not, transform the grammar to LL(1)

b) Compute First & Follow for the productions. Draw the parse table

a) No, it is left recursive. This grammar is supposed to be the language for balanced parentheses. There is a similar grammar that balances equal number of 0's & 1's, namely

$$S \rightarrow 0S1S \mid 1S0S \mid \epsilon$$

So ~~analogous~~ a similar grammar for parentheses is

$$S \rightarrow (S)S \mid \epsilon$$

b)

$$\text{First}(S) = \{ (, \epsilon \}$$

$$\text{Follow}(S) = \{ \#,) \}$$

	(	)	#
S	①	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

$$\textcircled{1} S \rightarrow (S)S$$

$$\textcircled{2} S \rightarrow \epsilon$$

3) The following grammar for regular expressions over symbols a & b only:

$$\begin{aligned} \langle r_{\text{expr}} \rangle &\rightarrow \langle r_{\text{expr}} \rangle + \langle r_{\text{term}} \rangle \mid \langle r_{\text{term}} \rangle \\ \langle r_{\text{term}} \rangle &\rightarrow \langle r_{\text{term}} \rangle * \langle r_{\text{factor}} \rangle \mid \langle r_{\text{factor}} \rangle \\ \langle r_{\text{factor}} \rangle &\rightarrow \langle r_{\text{factor}} \rangle * \mid \langle r_{\text{primary}} \rangle \\ \langle r_{\text{factor}} \rangle &\rightarrow a \mid b \end{aligned}$$

- left factor the grammar (recursion instead?)
- Does left factoring(?) make this grammar suitable for top-down parsing?
- What is the additional step necessary to make the grammar LL(1)? Fix the grammar?
- Compute First & Follow for the productions. Draw the parse table.
- Write the pseudocode for recursive implementation of LL(1) grammar.

$$\begin{aligned} \langle r_{\text{expr}} \rangle &\rightarrow \langle r_{\text{expr}} \rangle + \langle r_{\text{term}} \rangle \mid \langle r_{\text{term}} \rangle & \Rightarrow \langle r_{\text{expr}} \rangle &\rightarrow \langle r_{\text{term}} \rangle \langle r_{\text{expr}}' \rangle \\ & & \langle r_{\text{expr}}' \rangle &\rightarrow + \langle r_{\text{term}} \rangle \langle r_{\text{expr}}' \rangle \mid \epsilon \\ \langle r_{\text{term}} \rangle &\rightarrow \langle r_{\text{term}} \rangle \langle r_{\text{factor}} \rangle \mid \langle r_{\text{factor}} \rangle & \Rightarrow \langle r_{\text{term}} \rangle &\rightarrow \langle r_{\text{factor}} \rangle \langle r_{\text{term}}' \rangle \\ & & \langle r_{\text{term}}' \rangle &\rightarrow \langle r_{\text{factor}} \rangle \langle r_{\text{term}}' \rangle \mid \epsilon \\ \langle r_{\text{factor}} \rangle &\rightarrow \langle r_{\text{factor}} \rangle * \mid \langle r_{\text{primary}} \rangle & \Rightarrow \langle r_{\text{factor}} \rangle &\rightarrow \langle r_{\text{primary}} \rangle \langle r_{\text{factor}}' \rangle \\ & & \langle r_{\text{factor}}' \rangle &\rightarrow * \langle r_{\text{factor}}' \rangle \mid \epsilon \\ \langle r_{\text{factor}} \rangle &\rightarrow a \mid b & \langle r_{\text{factor}} \rangle &\rightarrow a \mid b \\ \langle r_{\text{primary}} \rangle & & \langle r_{\text{primary}} \rangle & \end{aligned}$$

$$\begin{aligned} \text{First}(r_{\text{expr}}) &= \{a, b\} \\ \text{First}(r_{\text{expr}}') &= \{+, \epsilon\} \\ \text{First}(r_{\text{term}}) &= \{a, b\} \\ \text{First}(r_{\text{term}}') &= \{a, b, \epsilon\} \\ \text{First}(r_{\text{factor}}) &= \{a, b\} \\ \text{First}(r_{\text{factor}}') &= \{*, \epsilon\} \\ \text{First}(r_{\text{primary}}) &= \{a, b\} \end{aligned}$$

$$\begin{aligned} \text{Follow}(r_{\text{expr}}) &= \{\$, *\} \\ \text{Follow}(r_{\text{expr}}') &= \{\$, \} \\ \text{Follow}(r_{\text{term}}) &= \{+, \$, \} \\ \text{Follow}(r_{\text{term}}') &= \{+, \$, \epsilon\} \\ \text{Follow}(r_{\text{factor}}) &= \{a, b, +, \$, \epsilon\} \\ \text{Follow}(r_{\text{factor}}') &= \{a, b, +, \$, \epsilon\} \\ \text{Follow}(r_{\text{primary}}) &= \{*, a, b, +, \epsilon\} \end{aligned}$$

- ① $\{rexpr\} \rightarrow \{rterm\} \{rexpr\}$
- ② $\{rexpr\} \rightarrow \epsilon + \{rterm\} \{rexpr\}$
- ③ $\{rterm\} \rightarrow \{rfactor\} \{rterm\}$
- ④ $\{rterm\} \rightarrow \epsilon + \{rfactor\} \{rterm\}$
- ⑤ $\{rfactor\} \rightarrow \{rprimary\} \{rfactor\}$
- ⑥ $\{rfactor\} \rightarrow \epsilon + \{rfactor\}$
- ⑦ $\{rprimary\} \rightarrow a | b$
- ⑧ $\{rexpr\} \rightarrow \epsilon$
- ⑨ $\{rterm\} \rightarrow \epsilon$
- ⑩ $\{rfactor\} \rightarrow \epsilon$

③)

	a	b	+	#	\$
$\{rexpr\}$	①	①			
$\{rexpr\}$			②		⑧
$\{rterm\}$	③	③			
$\{rterm\}$	④	④	④		⑨
$\{rfactor\}$	⑤	⑤	⑤		⑩
$\{rfactor\}$	⑩	⑩	⑩	⑥	⑩
$\{rprimary\}$	⑦	⑦			

```

② parseRexpr() {
    lookahead = getLookahead();
    if (lookahead == 'a' or lookahead == 'b') {
        if (lookahead == 'a')
            match('a');
        else
            match('b');
        parseRexprP();
    } else {
        error();
    }
}

```

```

# parseExprP() {
    lookahead = getLookahead();
    if (lookahead == '+') {
        match('+');
        parseRterm();
        parseRexprP();
    } else if (lookahead == '$') {
        return;
    } else {
        error();
    }
}

```



```

parseTerm() {
    lookahead = getLookahead();
    if (lookahead == 'a' or lookahead == 'b') {
        if (lookahead == 'a')
            match('a');
        else
            match('b');
        parseRterm();
    } else {
        error();
    }
}

```

```

parseTermP() {
    lookahead = getLookahead();
    if (lookahead == 'a' or lookahead == 'b') {
        if (lookahead == 'a')
            match('a');
        else {
            match('b');
            parseTermP();
        }
    } else if (lookahead == '+' or lookahead == '#') {
        if (lookahead == '+')
            match('+');
        else
            match('#');
        return;
    } else
        error();
}

```